Learning Diary

NodeJS and MongoDB crash courses – tutorials

22.10.2026

I learned what Node actually is from a theoretical standpoint, when it is usually used, and why. Node allows for code to run asynchronously thanks to its “event loop” function. It’s used for building APIs, server-rendered apps, real-time applications (chat, games, cooperation tools, etc.), bots, and many more, but it’s not good for CPU-insensive applications. I also followed the nodejs code tutorial and learned about its capabilities, common js syntax versus ES modules. I have previously taken the course “Advanced Web Applications”, so I was already familiar with these concepts, but my previous knowledge mainly focused on Express practices, so this tutorial not only refreshed my memory and gave a bit more context to the functions/modules I had been using (e.g., nodemon, environment variables, gitignore, async/promise use), but it also taught me a more traditional approach to developing node servers. I found it interesting how much more difficult the syntax becomes when using regular js syntax matching instead of express modules. I also used ‘os’, ‘crypto’ and ‘events’ (eventmeter) modules which were new to me.

Again, because I have taken the Advanced Web Applications course, I was already accustomed to building more complex websites, especially using MongoDB as my primary database implementation. The mongoDB crash course was mainly a tool for me to remember essential syntax for handling data, collections and filtering, as well as a tutorial for using Mongo Atlas service for holding and hosting the database instead of only hosting/storing it locally. This course had a lot more insightful details related to Mongo syntax, such as using the “$” ($set/$inc) notations for updating rubrics inside an entry, as well as different more advanced filtering options than I was used to. Aside from this, I also learned about aggregations, how stage order influences results and what types of stages are already provided; Everything was done within the GUI implementation of MongoDB (MongoDB Compass). When finally connecting MongoDB to a Node application in VS Code, I noticed the template code differs, so I switched to module instead of commonjs, and also used `.env` to store the URI, so that the Atlas Cluster password wouldn’t be published to github. The function calling differs a bit from the tutorial, mostly due to the code template provided by Atlas, but overall it does the exact same thing.

ExpressJS crash course

02.11.2026

Today I started the Express tutorial. I was quite accustomed to using the Express framework from previous experience, but now I'm getting a bit more background and context. For instance, I didn't know about the opinionated/unopinionated options,

their meanings (features vs. freedom) and use cases. Another very useful tool I found out about was the 'colors' npm dependency, which helps a lot with tracking logs in the console. I also learned about ejs templates as a useful page rendering tool which can pass dynamic data, which seems to be quite similar to PHP.

React

02.11.2026

I already knew about React as a front end development tool quite a lot, and the crash course presented just provided a bit more technical information, as well as interesting comparisons with other frameworks line Angular and Vue. React uses something called JSX which enables dynamic html and the apps using React have highly interactive Uis with virtual DOMs (where the reloading of the page doesn't happen in browser, but in the background, improving the user experience). React components can be created in different ways (classes vs functions), but I am more accustomed to the function definition. My previous knowledge in React was quite shallow, so the new material concerning states and their importance, as well as what hooks are from a more introspective view made my understanding process slightly more challenging. Unfortunately, since the crash course version of React is outdated, some syntax is no longer working (e.g, defaultProps). My previous experience with React was through Vite, so I tried following the concepts of the crash course video as closely as possible while using Vite as a substitute for CRA. Therefore, the code gets a lot more simple, but the file types switch from js to jsx, but the syntax is very similar if not the same. This was the first time I have tried out the build version instead of the development run. I also learned a bit about using `json-server` as a placeholder for a real backend service to see how a frontend communicates with an API.

MERN-Stack

03.11-04.11.2026

For the full stack implementation of a web application that accounts for users and their goals' management, I followed the 4-step tutorials presented in the course. So, I used everything I learned previously: MongoDB, Express, Node.js, and React. My favorite part was the backend, as I find routing very simple and fun. Logging in and registrations were also not too difficult for me to understand and put into practice, mostly thanks to my previous experience, and using JWT, which I was already comfortable with, made the whole process much quicker. However, the most challenging part was for me related to the Redux Toolkit. This was the first time I had heard of it and how it helps manage states and store shared application data in a central place instead of passing data between React components. Even though, I could somewhat grasp the concept from a theoretical standpoint, I found it challenging actually putting it into practice. In this project, Redux was used to store

authentication and goals information as well as states. From my understanding, we use slices for containing both the states and the rules for changing them. The concept of ‘Thunk’ was also something quite new to me, mostly because of its tight relation to Redux. The tutorial itself didn’t quite clarify thunks for me, as I initially thought that they act as a logic barrier between the frontend and the backend, but upon a bit of research online, I realised that they only run in the frontend, and that they are only an organisational layer between React components and API calls. Also, I learned that the reason we even use them is that Redux reducers are supposed to update states immediately, without making API requests themselves, so async thunks process API calls in the background instead. It’s very different from what I was used to, as I used to store authentication information in the localStorage instead, and most of my necessary states were local, so I didn’t need Redux at all, but rather React Context.

10.11.2026

Finally, I managed to deploy the application to Heroku, as I was initially having some trouble with running it from the git repository. I might have mistakenly changed one of the ‘.env’ variables when placing them into the Heroku app, but upon restarting the whole creation process, it eventually worked.